Method Lookup Revisited

In *What Happens When You Call a Method?, on page 27*, you learned about the Ruby object model and method lookup. Back then, we had to leave some parts of the object model unexplored. Singleton classes are the missing link we needed. Once you understand singleton classes, all the bits and pieces in the object model finally fall into place.

Method Lookup Review

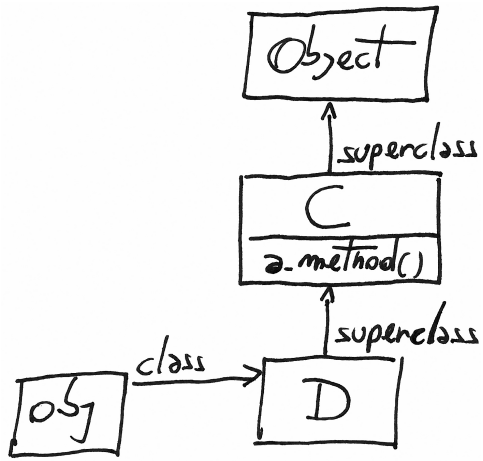
To look into the object model, you need a practical example to focus on. Let's write a "lab rat" program:

```
class C
  def a_method
    'C#a_method()'
  end
end

class D < C; end

obj = D.new
obj.a_method    # => "C#a_method()"
```

If you draw a picture of `obj` and its ancestors chain, it will probably look like the following figure. (For now, you don't have to bother with singleton classes or modules.)



You know that method lookup goes one step to the right, then up. When you call `obj.a_method()`, Ruby goes *right* into `obj`'s class `D`. From there, it climbs *up* the ancestors chain until it finds `a_method` in class `C`. Now, let's add singleton classes to the mix.